

CS 350 Milestone Two GPIO UART Lab Guide

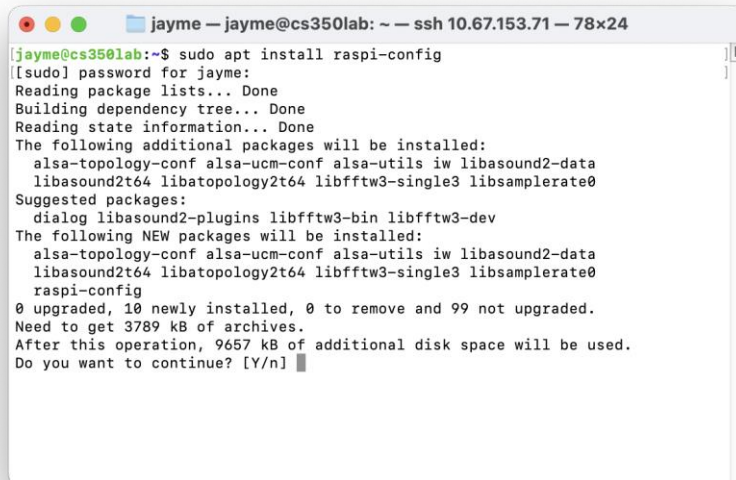
As you have been learning in your textbook, the UART is responsible for providing effective serial communication to an embedded device. In its default configuration, the Raspberry Pi's serial port runs the system console (login terminal) so that you can attach through the serial line, log in, and manage the device.

For our purposes, we are going to disable the serial port's terminal function because we will be utilizing it to represent commands that may be sent to our embedded device from a higher-level control system.

Step 1: Install raspi-config

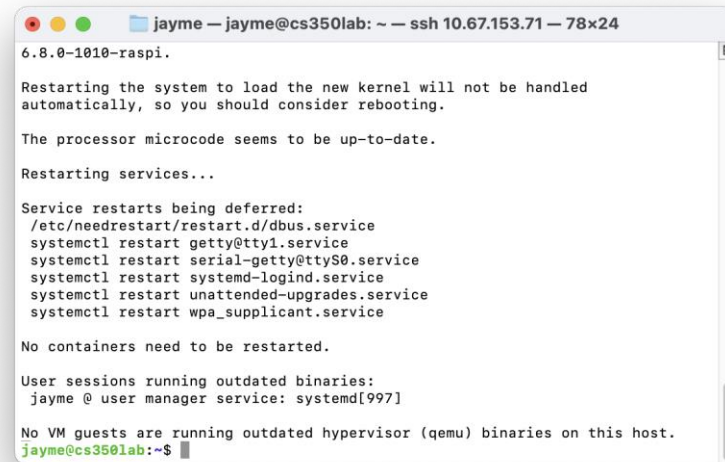
The first step is to install some additional software, raspi-config, to make some configuration changes to how the default operating system configuration works. Do this with the following command:

```
sudo apt install raspi-config
```



```
jayme — jayme@cs350lab: ~ — ssh 10.67.153.71 — 78x24
[jayme@cs350lab:~]$ sudo apt install raspi-config
[sudo] password for jayme:
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
The following additional packages will be installed:
  alsa-topology-conf alsa-ucm-conf alsa-utils iw libasound2-data
  libasound2t64 libatopology2t64 libfftw3-single3 libsamplerate0
Suggested packages:
  dialog libasound2-plugins libfftw3-bin libfftw3-dev
The following NEW packages will be installed:
  alsa-topology-conf alsa-ucm-conf alsa-utils iw libasound2-data
  libasound2t64 libatopology2t64 libfftw3-single3 libsamplerate0
  raspi-config
0 upgraded, 10 newly installed, 0 to remove and 99 not upgraded.
Need to get 3789 kB of archives.
After this operation, 9657 kB of additional disk space will be used.
Do you want to continue? [Y/n]
```

Enter Y, and press **return** when prompted.



```
6.8.0-1010-raspi.

Restarting the system to load the new kernel will not be handled
automatically, so you should consider rebooting.

The processor microcode seems to be up-to-date.

Restarting services...

Service restarts being deferred:
/etc/needrestart/restart.d/dbus.service
systemctl restart getty@tty1.service
systemctl restart serial-getty@ttyS0.service
systemctl restart systemd-logind.service
systemctl restart unattended-upgrades.service
systemctl restart wpa_supplicant.service

No containers need to be restarted.

User sessions running outdated binaries:
jayme @ user manager service: systemd[997]

No VM guests are running outdated hypervisor (qemu) binaries on this host.
jayme@cs350lab:~$
```

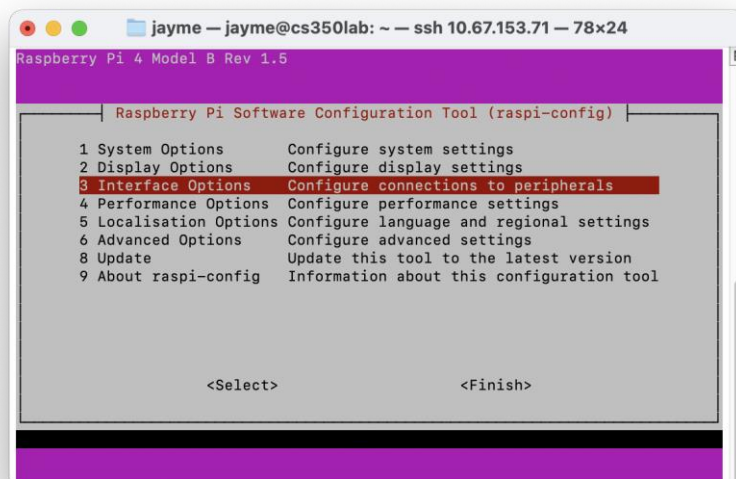
You should see a dialog similar to the above image when the installation has been completed.

Step 2: Disable Console on Serial Port

The next step is to use the `raspi-config` application to disable the console on the serial port and enable the UART for programmatic serial connection. You will start the configuration application with the following command:

```
sudo raspi-config
```

Use the arrow keys to select Option 3 as depicted in the following screenshot, and then press **TAB** to highlight the `<Select>` option and press **return**.



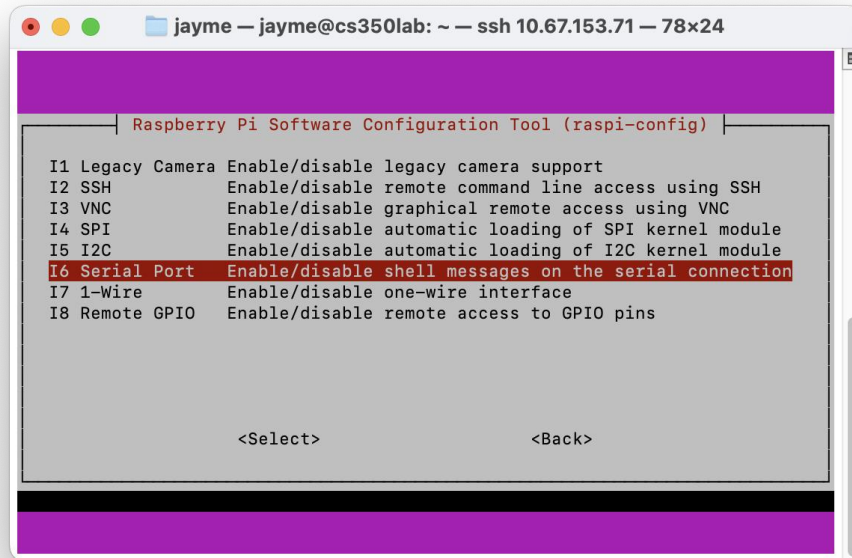
```
Raspberry Pi 4 Model B Rev 1.5

Raspberry Pi Software Configuration Tool (raspi-config)

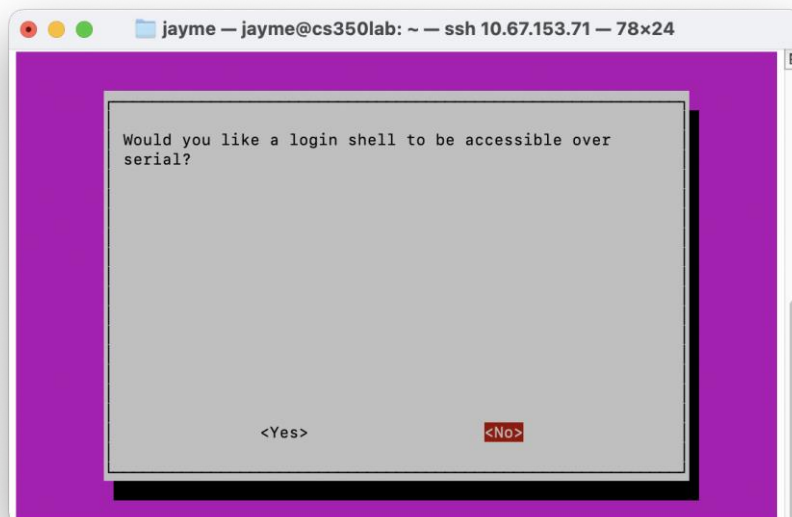
1 System Options      Configure system settings
2 Display Options     Configure display settings
3 Interface Options   Configure connections to peripherals
4 Performance Options Configure performance settings
5 Localisation Options Configure language and regional settings
6 Advanced Options    Configure advanced settings
8 Update              Update this tool to the latest version
9 About raspi-config  Information about this configuration tool

<Select>              <Finish>
```

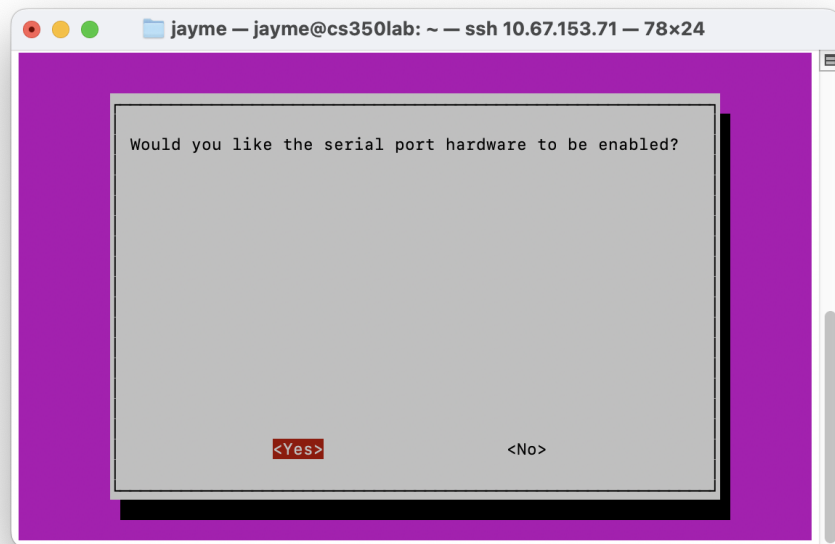
In the next screen, you will use the arrow keys to select I6 (the serial port option) and use the **TAB** key again to choose the `<Select>` option and press **return**.



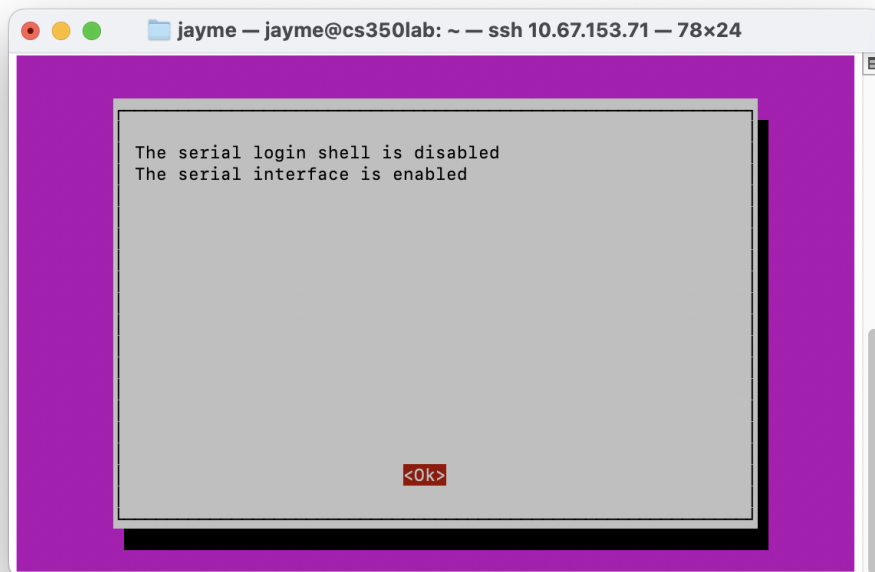
The next step is to answer the question: "Would you like a login shell to be available over serial?" Select the <No> option here.



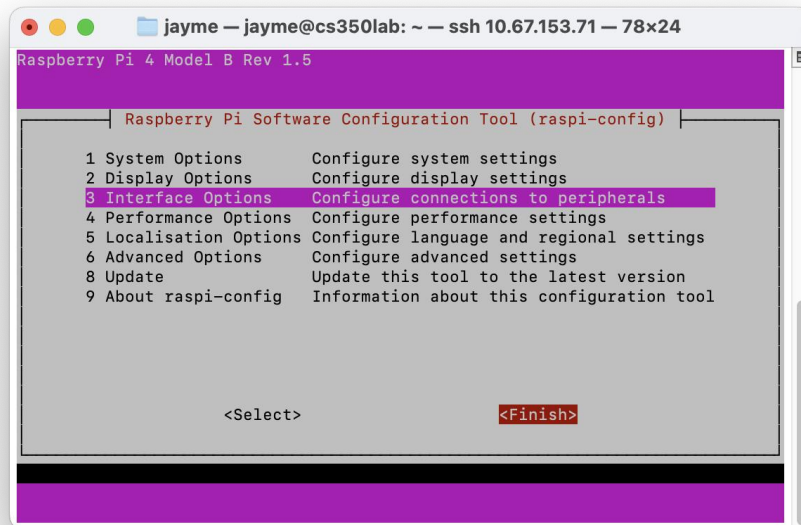
Next, verify that you want to enable the serial port hardware (UART). Select the <Yes> option for this.



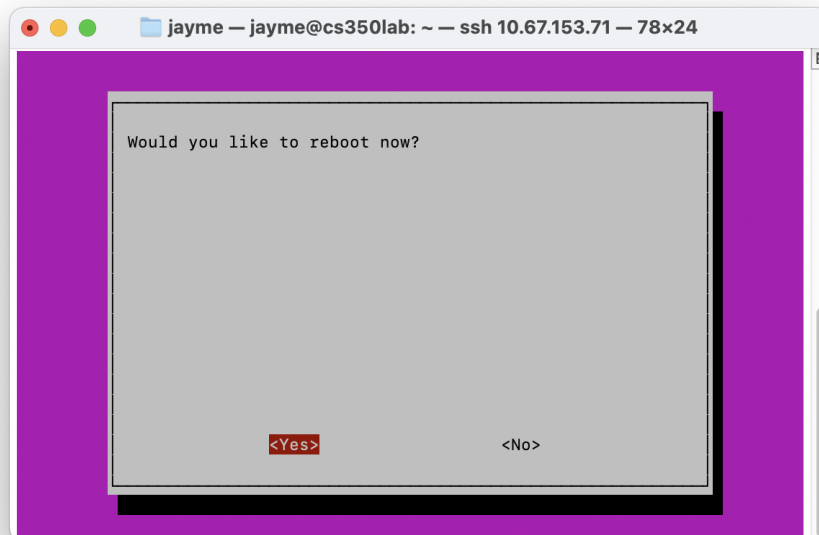
If everything is successful, you should see the notification that the serial login shell has been disabled and the serial interface enabled.



Select <Ok> to acknowledge the changes. This will take you back to the initial raspi-config screen, where you can select <Finish> to exit the configuration.



Because you have made fundamental changes to how the hardware will be handled, you will need to reboot your Raspberry Pi, which can be done automatically by selecting <Yes> on the final screen of raspi-config.



As the application exits, it will immediately process the reboot and close your terminal window.

Note: If you are working on the console with a keyboard and monitor attached to your Raspberry Pi, you will see the reboot sequence execute.



```
jayme — jayme@cs350lab: ~ — zsh — 78x24
[jayme@cs350lab:~]$ sudo raspi-config
Broadcast message from root@cs350lab on pts/0 (Sat 2024-09-07 22:40:24 EDT):

The system will reboot now!

[jayme@cs350lab:~]$ Connection to 10.67.153.71 closed by remote host.
Connection to 10.67.153.71 closed.
jayme@sapling-08 ~ %
```

Please wait a few minutes and log back in to ensure the system is reachable.

Step 3: Power Off Your Raspberry Pi

Your Raspberry Pi must be shut down and unplugged to continue. To safely shut down the operating system running on your Raspberry Pi, enter the following command from a terminal (or the console) of the Raspberry Pi:

```
sudo halt
```

You will have to enter your password as you are executing the halt command with root privileges by calling it with *sudo*. Wait at least two minutes after your terminal window closes before unplugging your Raspberry Pi.

Step 4: Connect the USB-to-TTL Cable

Components

- Assembled Raspberry Pi
- 3 x male-to-male jumper cables
- USB-to-TTL cable

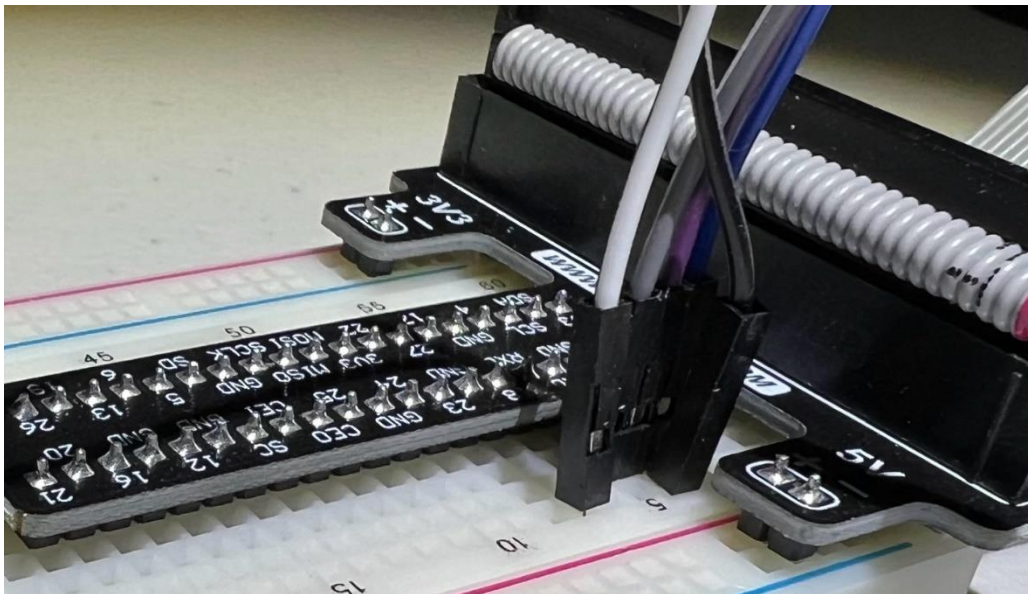




Connect the first jumper to Column B, Row 4 (ground) on your solderless breadboard. The other end of this jumper should be plugged into the black lead on the TTL cable.

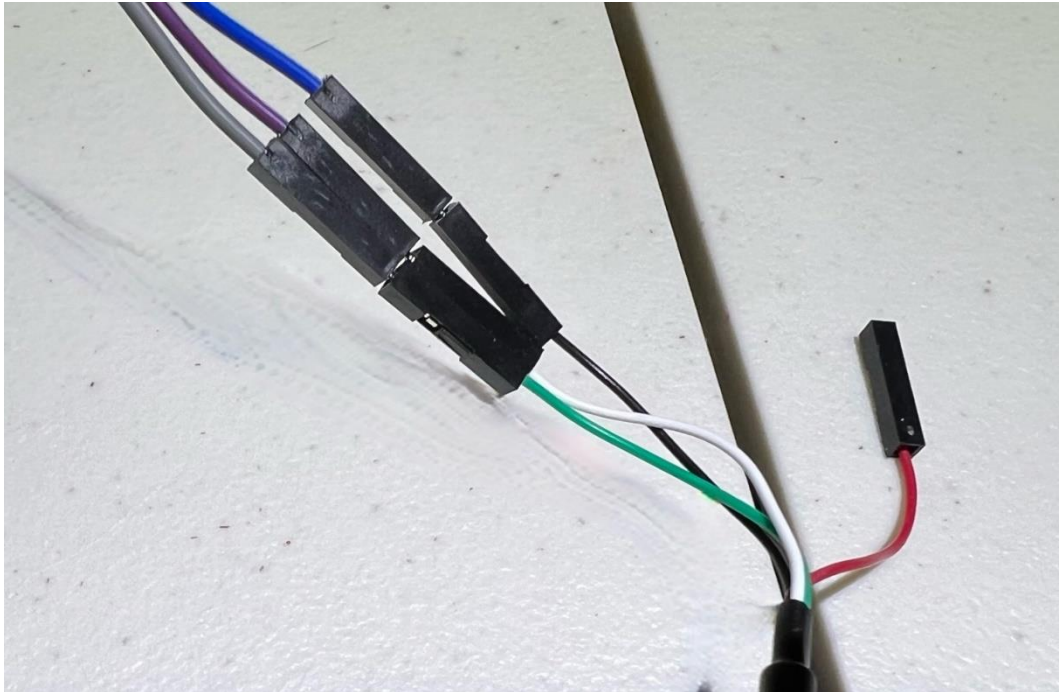
Connect the second jumper to Column B, Row 5 (TXD) on your solderless breadboard. The other end of this jumper should be plugged into the white lead on the TTL cable.

Connect the third and final jumper to Column B, Row 6 (RXD) on your solderless breadboard. The other end of this jumper should be plugged into the green lead on the TTL cable.



Note: In this view, the connections are behind the connections for your LED circuit.

Warning: Do not connect the red lead on the TTL cable!



Connect the USB end of the USB-to-TTL cable to a USB port next to the network jack on the Raspberry Pi.

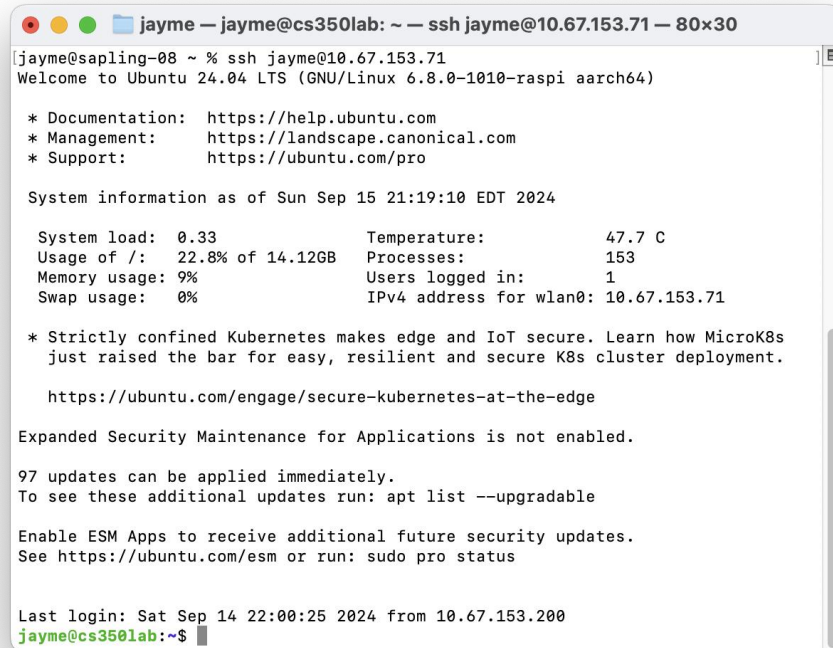


Step 5: Power Up Your Raspberry Pi

Plug in your Raspberry Pi. When it has completely booted, log in to the Raspberry Pi.

Step 6: Verifying the Serial Connection

This step requires you to have established two separate sessions on your Raspberry Pi. You can do this by opening two separate windows on your computer and using the **ssh** command to connect each to the Raspberry Pi.



```
jayme — jayme@cs350lab: ~ — ssh jayme@10.67.153.71 — 80x30
jayme@sapling-08 ~ % ssh jayme@10.67.153.71
Welcome to Ubuntu 24.04 LTS (GNU/Linux 6.8.0-1010-raspi aarch64)

* Documentation:  https://help.ubuntu.com
* Management:    https://landscape.canonical.com
* Support:       https://ubuntu.com/pro

System information as of Sun Sep 15 21:19:10 EDT 2024

System load:  0.33           Temperature:   47.7 C
Usage of /:   22.8% of 14.12GB Processes:    153
Memory usage: 9%           Users logged in: 1
Swap usage:   0%           IPv4 address for wlan0: 10.67.153.71

* Strictly confined Kubernetes makes edge and IoT secure. Learn how MicroK8s
  just raised the bar for easy, resilient and secure K8s cluster deployment.

https://ubuntu.com/engage/secure-kubernetes-at-the-edge

Expanded Security Maintenance for Applications is not enabled.

97 updates can be applied immediately.
To see these additional updates run: apt list --upgradable

Enable ESM Apps to receive additional future security updates.
See https://ubuntu.com/esm or run: sudo pro status

Last login: Sat Sep 14 22:00:25 2024 from 10.67.153.200
jayme@cs350lab:~$
```

You will need to do this twice because you will be testing writing to the serial port and reading from the serial port separately.

In the first window, you will want to run the SerialTest-Write.py script with the following command:

```
python3 SerialTest-Write.py
```

```
jayme — jayme@cs350lab: ~ — ssh jayme@10.67.153.71 — 80x30
Welcome to Ubuntu 24.04 LTS (GNU/Linux 6.8.0-1010-raspi aarch64)

* Documentation:  https://help.ubuntu.com
* Management:    https://landscape.canonical.com
* Support:       https://ubuntu.com/pro

System information as of Sun Sep 15 21:19:10 EDT 2024

System load:  0.33           Temperature:   47.7 C
Usage of /:   22.8% of 14.12GB Processes:    153
Memory usage: 9%           Users logged in: 1
Swap usage:  0%           IPv4 address for wlan0: 10.67.153.71

* Strictly confined Kubernetes makes edge and IoT secure. Learn how MicroK8s
  just raised the bar for easy, resilient and secure K8s cluster deployment.

https://ubuntu.com/engage/secure-kubernetes-at-the-edge

Expanded Security Maintenance for Applications is not enabled.

97 updates can be applied immediately.
To see these additional updates run: apt list --upgradable

Enable ESM Apps to receive additional future security updates.
See https://ubuntu.com/esm or run: sudo pro status

Last login: Sat Sep 14 22:00:25 2024 from 10.67.153.200
jayme@cs350lab:~$ python3 SerialTest-Write.py
```

You will notice that there is no output. That is because the application is writing a single line to the serial port every two seconds and incrementing the counter. You can see the results of this by running the corresponding SerialTest-Read.py script with the following command:

```
python3 SerialTest-Read.py
```

In the second window on the Raspberry Pi. You will see something that starts to look like this (your numbers will vary depending on when you started the SerialTest-Write.py script):

```
jayme — jayme@cs350lab: ~ — ssh 10.67.153.71 — 80x24
jayme@cs350lab:~$ python3 SerialTest-Read.py
Write counter: 127
Write counter: 128
Write counter: 129
Write counter: 130
Write counter: 131
```

The output will continue until you stop the SerialTest-Write.py script by pressing CTRL-C in the window running the script SerialTest-Write.py script. The window showing the output will then hang until it is interrupted by CTRL-C or until you restart the SerialTest-Write.py script.

Step 7: Explanation of the Connections

Examining each of the SerialTest scripts, you will see different headers for the serial connection. The SerialTest-Write.py script uses the ttyUSB0 device to write to the Raspberry Pi's serial port, while the SerialTest-Read.py script uses the ttyS0 device to read from the serial port on the Raspberry Pi. This shows a complete communication path through the USB-to-TTL cable on the Raspberry Pi.

Step 8: Changing the Code

The next step is to work with the code in the SerialLightControl-Server-Template.py to make some small changes to the template to handle commands coming in from the SerialLightControl-Client.py script. Similar to the SerialTest-Read and SerialTest-Write scripts, you will run the SerialLightControl-Client.py in one terminal, and once you have added your code to the SerialLightControl-Server.py file, you will run the code in a second terminal.

You will look for the sections of the file labeled **TODO** and follow the instructions to add the necessary code to complete the task. You will add code to handle four action commands that the client would send: ON, OFF, EXIT, or QUIT.

Hint: You can review the testing scripts to identify how the tests are performed and infer what you need to add to complete the code.

Lab Questions

Answer these questions in a separate Word document and include it as part of your submission for Milestone Two:

1. Why do both the SerialTest-Write.py and SerialLightControl-Client.py scripts use the encode() method of the string datatype when writing data to the serial port?
2. Why does the SerialTest-Read.py script use the decode() method of the string datatype when reading the data from the serial port?
3. What is the purpose of the try/except block in both the SerialLightControl-Client.py script and the SerialLightControl-Server.py script?
4. Why is it necessary to make sure that the GPIO pins are always returned to their original state at the end of program run?